

Kubernetes Pod CrashLoopBackOff Resolution SOP

Objective

Restore workload stability when a Kubernetes pod repeatedly restarts with CrashLoopBackOff due to configuration errors, runtime failures, or missing dependencies.

Scope

Applicable to Kubernetes workloads, platform operations, DevOps teams, SRE teams, application teams, and synthetic incident workflows across development, staging, and production-like environments.

Pre-requisites

1. Access to Kubernetes cluster using kubectl and the correct namespace context.
2. Access to application, pod, deployment, service, ingress, and node logs where applicable.
3. Access to monitoring dashboards such as Grafana, Prometheus, cloud monitoring, or container insights.
4. Permission to inspect manifests, secrets, configmaps, RBAC bindings, PVCs, and rollout history.
5. Escalation contact for DevOps, platform, infrastructure, network, and application teams.

Incident Metadata

Field	Value
Ticket ID	T002
Issue	Kubernetes pod crash loop
Category	Deployment
Service	Kubernetes
Severity	Critical
Status	Resolved
Description	Pod keeps restarting with CrashLoopBackOff
Expected Resolution	Check pod logs and fix config errors or missing dependencies

Procedure

1. Confirm the impacted namespace, deployment, pod name, restart count, and first failure time.
2. Run kubectl describe pod to review container state, exit code, events, probes, and mounted resources.
3. Check container logs and previous container logs to identify the crash reason.
4. Validate required environment variables, secrets, configmaps, volumes, image tag, and command arguments.
5. Check readiness and liveness probe configuration for incorrect path, port, or timeout settings.
6. Apply the required config, dependency, or manifest fix through approved deployment process.
7. Restart or rollout the workload and confirm pod status becomes Running and Ready.
8. Monitor restart count, app logs, and service availability before closing the ticket.

Common Commands

```
kubectl config current-context
kubectl get pods -n <namespace> -o wide
kubectl describe pod <pod-name> -n <namespace>
kubectl logs <pod-name> -n <namespace> --tail=100
kubectl get events -n <namespace> --sort-by=.lastTimestamp
kubectl get deploy,svc,ingress,pvc -n <namespace>
```

Troubleshooting Matrix

Issue	Possible Cause	Resolution
Pod CrashLoopBackOff	Application exits on startup	Review logs and fix startup failure
Container restarts repeatedly	Missing config or secret	Correct environment variables, secret, or configmap
Probe failures	Incorrect liveness/readiness probe	Fix probe path, port, and thresholds
Image starts locally only	Runtime dependency missing	Rebuild image with required dependency

Best Practices

1. Use readiness probes to avoid routing traffic to unhealthy pods.
2. Keep startup dependencies explicit and validated.
3. Store config in managed configmaps and secrets.
4. Alert on high restart count and CrashLoopBackOff events.

References

[Kubernetes Docs](#) | [Internal Tickets CSV](#) | [Cluster Events](#) | [Monitoring Dashboards](#) | [Application Logs](#) | [Platform Runbook](#)